

Securing SDN/NFV-Enabled Campus Networks with Software-Defined Perimeter-Based Zero-Trust Architecture

Francis A. Ruambo^{a,b,c,d,e,f,g,i}, Deqing Zou^{a,b,c,d,e,f}, Ivandro O. Lopes^{a,b,c,d,e,h}, Bin Yuan^{*,a,b,c,d,e,f,i}

^a*School of Cyber Science and Engineering, Huazhong University of Science and Technology, 430074, Wuhan, China*

^b*Hubei Key Laboratory of Distributed System Security, Wuhan, China*

^c*Hubei Engineering Research Center on Big Data Security, Wuhan, China*

^d*National Engineering Research Center for Big Data Technology and System, Wuhan, China*

^e*Services Computing Technology and System Lab, Wuhan, China*

^f*Cluster and Grid Computing Lab, Wuhan, China*

^g*Mbeya university of Science and Technology, Mbeya, 131, Tanzania*

^h*Nucleo Operacional para a Sociedade de Informacao, Cape Verde*

ⁱ*Songshan Laboratory, Zhengzhou, China*

^{*}*Corresponding Author: yuanbin@hust.edu.cn*

Abstract

Network softwarization is a breakthrough in designing modern networks and providing numerous new network operations and services. This change is exemplified by Software Defined Networks (SDN) and Network Function Virtualization (NFV). As a result, the combination of SDN and NFV (SDN/NFV) frameworks and individual SDN and NFV frameworks have gained substantial traction in network security over the past decade, significantly supporting on-demand protection measures. However, the inherent security shortcomings of SDN, NFV, and SDN/NFV frameworks provide the fundamental issue in implementing security procedures via network softwarization. To solve this issue, we present a Software-Defined Perimeter (SDP)-powered Zero-Trust Architecture (ZTA) to improve security in the SDN/NFV-enabled campus network holistically. In addition to this, we also propose a threat model and quantitatively assess the suggested approach's performance and security. Furthermore, we highlight potential zero-trust network security function implementations in SDN/NFV-enabled infrastructure to increase protection for endpoint devices, network resources, and flows for detecting threats and responding effectively with minimal latency and overhead.

Keywords: Access Control, Campus Networks, SDN/NFV, Software Defined Perimeter, Zero Trust Architecture

1. Introduction

Network softwarization keeps up with growing demands and challenges in providing and managing services in modern networks. For instance, “Bring Your Own Device” (BYOD), mobile clients, video, and the ever-increasing quantity of connected devices and running applications, including the Internet of Things (IoT), are swiftly shifting the network landscape, no matter whether the campus is educational or corporate[1]. Specifically, because campus networks are heterogeneous, they are frequently challenging to manage, leading to unnecessary extra costs, scalability, and reliability challenges [1, 2, 3, 4, 5, 6, 7, 8, 9]. Usually, network configuration modifications are subject to long provisioning times and configuration flaws because network devices must be configured separately, typically through proprietary element management systems or the Command Line Interface (CLI). Furthermore, modifications/changes are inevitable due to network services and infrastructure evolution, modifications/changes are inevitable, leading to more misconfigurations and more risks. In this regard, Network Function Virtualization (NFV) and Software-Defined Networking (SDN) as network softwarization technologies have emerged as enablers in alleviating network provisioning, configuration, and management challenges by providing flexibleness and the capacity to develop innovative capabilities swiftly and cost-effectively [10, 11, 12, 13, 14, 15]. The main motives for adopting SDN/NFV solutions are to overcome capital and operation expenditures (CAPEX and OPEX, respectively) and give quick conveyance of network function, dynamic service chaining, and elasticity [10, 16, 17]. Typically, NFV manages virtual middleboxes as virtual network functions (VNFs), while SDN offers interconnection between VNFs for network services provisioning as Service Function Chaining (SFC). The SFC allows the inclusion of new VNFs to a packet’s flow, such as a Trusted Module Platform (TPM), an Intrusion Detection System (IDS), or a firewall, to mention a few.

SDN and NFV have recently gained tremendous momentum, signifying key enablers for providing on-demand protection mechanisms. Within the SDN/NFV-enabled infrastructure, it is much easier to understand the flow context (including application, device, user, time, location, and other latent external factors). Hence, it empowers network administrators to configure and enforce the required fine-granular policies at either the access or intermediary switch[18, 19, 20, 21].

Furthermore, several proposed SDN/NFV frameworks offer security mechanisms with greater flexibility and less operational costs [22, 23, 24, 25, 26]. The flexibility and benefits of network softwarization bring efficient and scalable performance, but it can open up our networks to unnecessary risks if not done correctly. Even though efficient solutions are achievable through SDN and NFV, security vulnerabilities still exist [18, 19, 27, 28, 29, 30].

Specifically, the inherent security flaws of SDN and NFV, as highlighted and analyzed in [31, 32, 33, 34, 35, 36, 37] and [38, 39, 40, 41, 42, 43], respectively, impede the main challenge in the adoption of network virtualization security mechanisms. Within the SDN/NFV environment, five elements are potentially under consideration for compromising: VNF applications (including SDN switches as VNFs), SDN controllers, SDN applications, managers, and hypervisors. SDN switches accomplish similar network function tasks, providing a forwarding function from a logical viewpoint. In this sense, protecting SDN/NFV frameworks and infrastructure can be a great advantage against many attacks. These attacks include but are not limited to the well-known Denial of Service (DoS) and Distributed Denial of Service (DDoS). The impact of these attacks on network services varies from medium to high, such as unexpected service outages, traffic flow redirection to bypass security technologies, or steering traffic toward compromised systems, for the interception or modification of traffic elsewhere in the network [43].

Likewise, the NFV/SDN preferred solutions experience security issues linked to integrating the two concepts, for example, insufficient authorization and authentication techniques within the communication interfaces among NFV and SDN modules. To this end, it is essential to come up with (develop) a unified framework to tackle both SDN and NFV issues rather than focusing on addressing them separately. This kind of approach simplifies execution and reduces the learning and adaptation window, as well as decreases interdependencies (raised by the necessity of plugins necessary to facilitate various frameworks interconnections) [17, 44, 45]. To address this issue, we use a Software-Defined Perimeter (SDP) [46, 47, 48, 49] as an implementation approach Zero-Trust Architecture (ZTA) [50, 51, 52] to improve security holistically within the SDN/NFV-enabled campus network. Additionally, we highlight its efficiency and efficacy in thwarting the frequent attacks on the campus networks, as highlighted and analyzed in research works [53, 54, 55, 56, 57]. To this end, our contributions from this paper are fivefold:

1. We review zero-trust approach principles and concepts and describe how a

zero-trust approach can leverage existing inventory systems and infrastructure to enhance security within SDN/NFV-enabled campus networks.

2. We propose a ZTA-SDP framework to protect campus networks that support SDN / NFV functionalities.
3. We quantify and incorporate network agents' risk/trust score, aka ephemeral agents, to implement a trust algorithm for adaptive granular authorization decision-making per the resources/services access requests basis.
4. We analyze the proposed model's security against a provided threat model and evaluate the performance overhead introduced in adopting the proposed model.
5. We highlight the possibilities of incorporating the standard lightweight SDN/NFV-powered network security functionalities in a zero-trust compliant manner through several everyday campus use cases.

The organization in the remaining part of this paper is as follows. Section 2 presents the related work as state-of-the-art regarding strategy and mechanisms for improving Security in SDN/NFV-enabled campus networks. Section 3 describes the zero-trust architecture, principles, concepts, and the proposed secure SDN/NFV-enabled campus network approach. Section 4 provides a threat model and analyzes the proposed model's security and performance. Section 5 explains the operation mode and security functionalities incorporated in the proposed architecture through simple use cases. The last section concludes and presents future works.

2. Related Work

State of the art for enhancing security in campus networks incorporates many enabling technologies and strategies. This section reviews the security enhancement mechanisms for SDN/NFV-enabled campus network infrastructure separately in the two subsections. The first subsection focuses on measures or techniques that can be used to improve the security of the campus network, which is enabled explicitly by the SDN/NFV technologies. The second subsection focuses more on the security of the SDN/NFV-enabled network infrastructure beyond just campus networks.

2.1. Security Enhancement Mechanisms for SDN/NFV-Enabled Campus Networks

Networks are continuously evolving, and this continuous change drives both difficulties and opportunities for the network security position. The SDN and NFV

result from this evolution, among many other technologies. The flexibility and benefits of the SDN and NFV bring efficient and scalable performance through network service orchestration. Network service orchestration allows automated network behaviors that offer easy management and coordination of hardware and software elements in supporting services and applications. With a flexible architecture, network engineers can troubleshoot and deploy networks at a level that was impossible in the past. While network service orchestration does not actively stop security threats, it enhances its overall security position. For example, network service orchestration offers the following capabilities [58]:

- It avoids human errors linked to the network and operation configurations.
- It guarantees proper rollback of configurations and avoids inadequate configuration mechanisms such as ACLs.
- It enforces uniform policy across various vendor devices, checks compliance, and adapts policy to remove configuration drift.
- It collects audit history and keeps configuration backup.
- It enforces Role-Based Access Control (RBAC) at granular user, device, and policy levels and automates it.

There are significant research works done on enhancing security in campus networks through a myriad of best practice principles and approaches that focus on SDN/NFV techniques to satisfy security specifications in a given context [2, 59, 60, 61, 62, 63]. In [60], the authors identified problems with the current state-of-the-art network management and configuration mechanisms. They introduced new SDN mechanisms to improve the supervision of various network management aspects. Also, in [61] authors leverages the SDN framework to address security challenges in enterprise networks, such as user box spoofing and security middle boxes. By binding users and machines to defined network appliances and ports, the proposed SDN framework was able to provide a precise and secure solution. The research work [59] addressed the management and configuration problems using SDN by implementing an application to flexibly manage and troubleshoot Virtual Local Area Networks (VLANs). In [64], the SDN provides NFV solutions for campus networks, i.e., Captive Portal, SDN dynamic policy provisioning applications, and Network Access Control (NAC) applications. In research work [2], authors explored SDN viability in the campus network for

security and network analysis through network monitoring and attack detection simulation. The proposal, called OpenSec[63] was used in the campus network to devise and execute security policies written in human-readable language. It is a security framework based on OpenFlow that enhances security through policy-based network security functions. To gradually implement and control the traffic flow precisely and securely in a campus network, authors in [62] applied an OpenFlow-steered SDN architecture with commodity access switches. Applying the best practices, principles, and techniques in the literature mentioned above focuses on enhancing security within the campus networks through SDN/NFV network services orchestration and management capabilities. However, their security approaches are still perimeter-based as the primary focus is hardening security through addressing/patching a known vulnerability/attacking surface and management-related issues. Fundamentally, the perimeter-based model differs from the zero trust model.

On the one hand, the perimeter model strives to create a wall isolating untrusted and trusted resources (i.e., the Internet and local networks), monitoring and filtering the north-south traffic. On the other hand, the ZTA model admits that the threats are omnipresent. Rather than create walls to protect the delicate entities inside, it turns the entire network into an unsecured territory [51], which never trusts and always verifies all requests to access resources. The zero-trust idea transfers access controls from the network's edge to individual devices and their users, allowing them to work securely on any device anywhere and anytime without a VPN [65, 66, 67, 68]. Furthermore, the zero-trust approach is beyond network segmentation through the Next-Generation Firewall (NGFW) to enforce that segmentation. The zero-trust approach is more than just network segmentation; It is an entirely holistic approach, including the following processes and technologies: zero-trust data, zero-trust devices, zero-trust people, zero-trust networks, zero-trust workloads, automation and orchestration, visibility, and analytics [69].

From the networking perspective, our approach considers SDN/NFV technology advancements with the ZTA approach to utilize the same flexibility with a "security-first" mindset. In this regard, we incorporate inbuilt security attributes from a zero-trust approach and leverage network service orchestration and management capabilities of SDN/NFV environments to achieve our objectives. This synergy results in a valuable combination of flexibility and security that helps the SDN/NFV-enabled campus networks to holistically reduce the risks and attack surfaces.

2.2. Mechanisms for Securing SDN/NFV-Enabled Network Infrastructure

SDN and NFV are attaining significant drive, serving as critical enablers towards provisioning on-demand security mechanisms. However, it can open the network to unnecessary risk; SDN and NFV suffer inherent security flaws [18, 19, 70]. Furthermore, integrating the two paradigms into SDN/NFV increases complexities and security vulnerabilities. For instance, a compromised or malicious network application can utilize the programmability of SDN/NFV networks to disrupt various network services, destroy information confidentiality, and affect network behavior in various ways. In this sense, to protect the resources available in SDN/NFV networks, it is necessary to control each application's interactions with the infrastructure.

To this aim, several works of literature discuss valuable techniques for accurately monitoring the network infrastructure and remote virtual machines attesting towards a secure and trusted practice of SDN and NFV. For example, authors in [71] envisioned that software-based networks could be trusted if their predicted behavior was guaranteed. Hence, it discussed and showed how remote attestation could guarantee software and hardware integrity in distributed systems and its application to SDN/NFV-enabled infrastructure. The described method is an essential requirement to guarantee the integrity of the network. In the SDN/NFV context, an SDN verifier inspects the integrity of the OpenFlow rules computed by the NFV node under the SDN controller to verify the whole network's integrity state and issue an alert in case of node compromisation. However, this approach is less efficient than proactively avoiding node tampering or compromising.

Furthermore, the authors in [72] incorporated a policy-based approach to propose an integrated security architecture. It coordinates various security techniques to detect and mitigate security attacks in SDN/NFV-enabled infrastructure. Furthermore, it deals with dynamic shifts in security attacks and trust variations related to network devices. The flexibility of the suggested architecture to dynamic shifts was accomplished by possessing feedback between the multiple security elements and mechanisms within the proposed architecture and using a dynamic policy framework to manage them. Our approach seems similar to this one. However, we extend the approach above by incorporating quantified variable trust scores for decision-making on the network's resource access requests. Additionally, we cloak/blacken both the devices and the infrastructure (i.e., the SDN/NFV framework) from unauthorized requests. In this regard, we adopt SDP [47, 48, 49] as the enabling architecture for ZTA towards proper cloaking/blackening/hiding the inherent security flaws in the proposed secure SDN/NFV-enabled campus network. The related research works highlighted in the above subsections improved the

security of campus/enterprise networks and SDN/NFV frameworks. However, none altogether leverages a zero-trust approach to proactively address the inherent security flaws of SDN/NFV frameworks adopted in campus networks. Our suggested approach is a panacea toward a secure SDN/NFV framework for campus networks. To this end, we introduce a ZTA-SDP approach for a well-designed and maintained secure SDN/NFV framework for the campus network to prevent, detect and respond to the myriad of attacks to enhance the campus security posture. Our approach aims to reduce attack surfaces by cloaking the infrastructure and devices parallel with stringent dynamic access control mechanisms and containment of the network segment for an appropriate response if a breach occurs (to hamper an attacker's efforts to move laterally within a network for exfiltrating data).

3. Zero Trust Architecture

This section presents the principles and concepts of the zero-trust approach and describes SDP architecture as the strategy for implementing the zero-trust philosophy.

3.1. Zero Trust Architecture Approach

A ZTA strategy is the paradigm where no absolute trust is granted/given to systems based on the network's environment or location (i.e., the Internet vs. local area networks) [51, 52, 73]. In the ZTA approach, granting data resources access is per request, and authentication (both device and user) is mandatory before establishing the connection [51, 52, 73]. The ZTA responds to enterprise network trends that involve cloud-based assets and remote users whose locations are within a network boundary of an owned enterprise [51, 52, 73]. The ZTA converges on protecting resources; hence, the network location becomes a less significant component of the resource's security posture [51, 52, 73]. As a result of its control over endpoints in the network, the zero trust architecture approach expands upon the internet threat model to consider compromises at the endpoints [51, 52, 73]. The response to these threats is generally first to facilitate the detection of those compromises. Detection is possible through scanning devices and behavioral analysis of the activity from each device. Furthermore, mitigation of endpoint compromise is managed by regular upgrades to software on devices, regular and automatic credential rotation, and in some cases, frequent rotation of the devices themselves [51, 52, 73]. From this perspective, improving

an enterprise's IT security posture is possible through different ZTA use cases and deployment models.

3.2. *Zero Trust Architecture Concepts and Principles*

The ZTA is a corporate network security strategy that uses the concept of zero-trust philosophy and includes workflow planning, component relationships, and access strategies. An operative perspective of zero-trust architecture offers ideas, principles, and concepts to reduce ambiguity when making correct, on-demand access decisions on information systems and services amid a compromised network. In this regard, users, applications, and devices, by default, are not trusted; hence verification per-request access is always mandatory altogether with their provisioned least privilege access. The basic tenets of zero-trust architecture include the following [51, 52, 73]:

- Resources (i.e., all computing services and data sources) Secure communication regardless of network location
- The granting of access to particular resources within the enterprise is session-aware.
- The dynamic policy defines access to resources. It matches the noticeable state of the application, client identity, and the requesting asset—and may incorporate other behavioral characteristics.
- The enterprise guarantees that all owned and correlated devices are within the most appropriate secure state and monitors assets to maintain the most trustworthy posture possible. In addition, all resources' authentication and authorization are dynamic and rigorously enforced before granting access.
- The enterprise gathers as much information as plausible on the current communications and network infrastructure state and utilizes it to enhance its security posture.

To understand how ZTA accomplishes the concepts and principles mentioned above, we use the high-level ZTA conceptual framework in Figure 1 to explain the fundamental relationship between the logical components and their interactions. For more information, references[51, 52, 73] can be more insightful.

- Network Agent

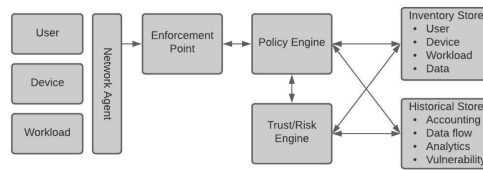


Figure 1: Zero-Trust Architecture Conceptual Framework

This network agent is an ephemeral entity with information about the access source for confidence calculation. Proper evaluation is easily achievable by including improved identity attributes such as asset and user status, content, authentication method, location, and trust scoring.

- Enforcement Engine

The Enforcement Engine (EE) refers to an Enforcement Point (EP). The EE authorizes the communication requested on a policy basis and continuously monitors the traffic to suspend it if required, as requested by the PE. The EE placement is typically nearly attainable to the secured element (the data). It acts as the data's guard. For instance, the EE may restrict a system holding the secured element from being discoverable.

- The Policy Engine

The Policy Engine (PE) is liable for the final decision to grant/deny access for a given subject to a resource. In this regard, the PE also serves as the Policy administrator (PA)[51], which means it is responsible for establishing and shutting down the communication path between a resource and a subject. It would create an authentication token, including client credentials for enterprise resources access. The PE applies enterprise policy, including input from outside sources (e.g., continuous diagnosis and monitoring systems, threat recognition services, et cetera) to a trust algorithm to deny, award, or revoke resource access. The PE executes the final decision to allow access to the assets/resources, logs the decision, and informs the EE. The policy rules depend on the technology implemented; however, they typically comprise the what, who, where, when, why, and how to get/grant access to network services, data classes, and endpoints. When creating the communication path, the PE communicates with the Policy enforcement point (the PEP) through the control plane.

- Policy Enforcement Point

The Policy Enforcement Point (PEP) is accountable for facilitating, monitoring, and ultimately ending connections amid an enterprise resource and a subject. The PEP comprises one logical part in ZTA, yet might be split into two unique parts as follows: the client (e.g., an agent on the user's laptop) and the one on the side of the resource (e.g., gateway component that controls access sits in front of the resource), or a gatekeeper for communication paths(aka, a single portal component). After the PEP is the implicit trust zone hosting the enterprise resource.

- Trust/Risk Engine

The Trust/Risk Engine examines the risk of an action or a request. The trust/risk Engine notifies the policy engine of divergences in a trust algorithm implemented, assesses the communication agent's data abreast data stores, and continually employs machine learning and static rules to update the agent trust scores. A trust algorithm implemented to calculate a score based on the confidence level derived using the enterprise's values, weights, criteria, contextual aspects of an agent's history, and other data gives the best and most comprehensive strategy for defeating threats. A contextual and score-based trust algorithm will recognize a threat or an attack that may reside inside a user's role, unlike an algorithm that does not consider historical information, including other user data. For instance, a contextual and score-based trust algorithm may detect a user account or role accessing data outside regular business hours from an unrecognizable location or in an unusual way. Another algorithm that relies only on a particular qualified set of attributes may assess faster but will not possess the historical meaning to learn that the access request seems unusual and notify the policy engine to ask for better authentication. Figure 2 illustrates an example of trust algorithm input.

- Data Stores

As stated before, a favored approach implements a contextual and score-based trust algorithm. This component holds different elements related to devices, users, workloads, and classified data correlated with these elements' historical data as an inventory store. Therefore, trust/risk and PEs must reference a stored data set to perform policy decisions against access requests, including communication behavior changes. To this end, leveraging behavior analytics will inform decision-makers to make suitable access decisions.

3.3. Zero Trust Architecture Implementation Strategies

ZTA is just an evolving security mindset or philosophical cybersecurity approach to improve security posture within a digital transformation and cyberspace.

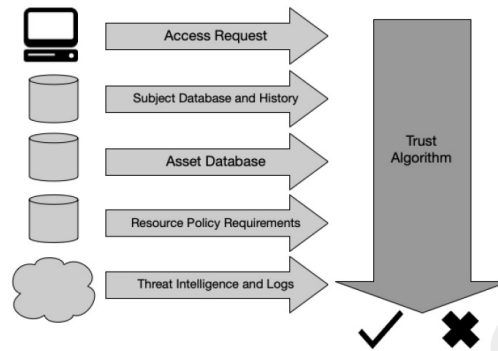


Figure 2: Trust Algorithm Input[51]

The joint projects to realize/implement ZTA are micro-segmentation and SDP. At first, this subsection gives a brief overview of micro-segmentation and SDP architectures from the ZTA perspective. Then, it emphasizes the role of the SDP architecture in zero-trust strategy implementation for securing network infrastructure, which is the main focus of this research work.

3.3.1. Micro-segmentation versus SDP in the context of ZTA implementation

Both micro-segmentation and SDP are currently trending projects in the journey of achieving a zero-trust strategy. On the one hand, micro-segmentation takes a zone-based approach like VLAN design; however, with micro-segmentation, further segments within a VLAN and apply concrete and fine-granular policies to protect the application environment. This approach allows to control and define rules for how applications can communicate. As a result, it allows a set of specific mapping rules (one-to-one) and limits the lateral movement within a zone if one actor/entity is compromised. It ensures that only every entity is doing what is only entitled and nothing else (least privilege in action). Instead of using Internet Protocol (IP) addresses, policies are tied to logical attributes such as identity, tag, or labels to check the payload upon certificate use. In micro-segmentation, continuous monitoring is mandatory for checking acceptable behavior baselines, such as regular traffic values. Therefore, it necessitates real-time and automated (as the manual approach is error-prone and inefficient) visibility on traffic and individual levels (i.e., process and data contextual level). Also, avoiding under-segmentation means that communication flows must cover all seven OSI model layers. An adequately segmented network can mitigate common network-based attacks, including port scanning, DoS, SQL injection, application vulnerability exploits, Man-

in-The-Middle attacks (MiTM), et cetera.

On the other hand, the SDP understands the evolving paradigm of no inside or outside concept from the users' perspective. Instead, the perimeter moves with the users, and users care only about access to the application. Hence, it adopts the need-to-know basis model to secure the application network infrastructure by establishing multiple granular user perimeters. In this regard, it hides assets in the infrastructure via the deny-all firewall/gateway until users/devices are proven. It also incorporates the lightweight UDP packet known as Single Packet Authorization (SPA) [46, 47, 49, 74, 75, 76, 77] to enable integrated controls for authentication and authorization. Furthermore, it follows the "Authenticate BEFORE Access" principle by implementing a separate control and data channel. Finally, it also mandates mutual TLS (mTLS) communication, meaning that it validates before TLS/TCP handshakes with the two-way mutually encrypted communications enforced in each communication.

In summary, SDP is a network security strategy that establishes an invisible perimeter around the network, making it difficult for attackers to access sensitive data or systems. Furthermore, it improves security by restricting network access to only approved people, devices, and applications. Micro-segmentation, on the other hand, focuses on breaking a network into smaller pieces and applying individual security policies to each segment, so minimizing the attack surface and protecting critical data. In this regard, SDP is a preferable alternative if the firm demands strict security, robust access control, and high secrecy for critical assets. Micro-segmentation, on the other hand, maybe a more viable choice if the company requires simpler, more flexible access control and does not require such strong confidentiality. Towards this end, we have opted for SDP as the zero-trust strategy implementation for securing the SDN/NFV-enabled campus networks. The following subsection describes the role of SDP architecture in zero-trust strategy implementation.

3.3.2. The Role of SDP Architecture in Zero-Trust Strategy Implementation

SDP refers to a connection-centric open security architecture that does not expose the infrastructure holistically [73, 74]. Instead, its operation adopts the need-to-know approach, which embraces security functionalities, including authentication, authorization, accounting, and system components such as users and devices before network infrastructure connection. The SDP setup connection differs from the standard TCP connection approach of connecting first and then authenticating. Unlike the TCP, the operation of SDP architecture first builds a bidirectional continuous trust between the protected application and the client before setting up

the connection. After establishing sufficient trust, it authenticates the client and securely connects to access the allowed resource/application.

Furthermore, the SDP leverages the SPA packet for authorization to create the invisible network infrastructure. The SPA is the lightweight protocol at the helm of (i.e., liable for) validating the devices or user identities before allowing network access via a gateway with the default-deny-all firewall setup[46]. In this way, It cloaks the infrastructure and its applications. Thus, it allows continuity of the processes associated with the network operation while patches and configurations for vulnerabilities can still be in progress. Since that SDP is connection-centric and not network-centric, each user on the specific device gets an individualized secure connection to a specific service hidden by the perimeter. Instead of just a single perimeter for the clients to use, SDP creates many small cryptographically signed micro-perimeters dedicated to clients purposely built to reflect the system components' context. SDP allows dynamic adjusting to changes within the network infrastructure by detecting/tracking the changes and applying appropriate security controls based on the network entities' context (i.e., applications, devices, and users). SDP architecture takes advantage of separating the data and control plane to decouple both the devices and users from the network. The control plane is liable for authentication first; then, the data plane via the client application connection facilitates data transfer, thus, providing the least privileged access rather than broad access. This way, it guarantees secure application access by the devices and users regardless of the network perimeter.

Implementing a zero-trust security model via SDP architecture involves leveraging the features and functionalities such as the SPA, mutual transport layer security(mTLS), device approving/validation, advanced/dynamic firewalls, and application session/request binding[73, 75]. For instance, unlike VPN implementation, which lacks fine-granular access control, the SDP operation provides absolute granular trust mechanisms (i.e., authorization, authentication, and access). The SDP also follows the authenticated and authorized before obtaining access paradigm, enhancing the protected infrastructure security posture by hiding it from unauthorized network queries and scan attempts. However, the instant authentication capability and separating the control and data plane are necessary to achieve the security mentioned above features and functionalities effectively. Furthermore, upon a failure in authentication, the SDP restricts network exposure and connectivity by dropping the network connections. Lastly, it facilitates monitoring suspicious activities by continuously tracking all connections and service usage.

Figure 3 illustrates the SDP architecture's primary components; its operation description is detailed hereafter. The primary architecture for essential SDP op-

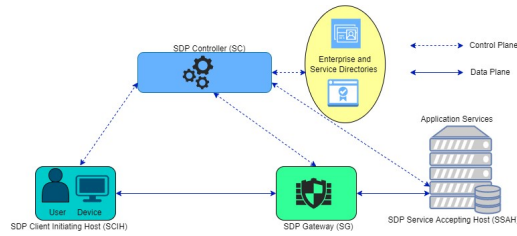


Figure 3: The SDP architecture depicting the primary system components

erations comprises the system components, including the SDP Client Initiating Host (SCIH), SDP Service Accepting Host (SSAH), SDP Gateway (SG), and SDP Controller (SC). The SCIH (i.e., the user and device) and SSAH (protected resource/application) are connected to SC, the authentication center, and the policy engine. The SC assigns the appropriate policy to the SCIH, determining/resolving its allowed resource access. The SC provides the control layer and allows authentication of SCIH and SSAH before allowing any communication. The SPA protects the SC and SSAH by limiting their accessibility and visibility to unauthorized network entities(i.e., users and devices). As a result, every network entity is cloaked/blackened/hidden from users. The SG sits before the resources to protect them, allowing only authorized network entities (i.e., devices and users) to access the secured services and processes. Once the SC grants the policy to the CISH, the SG opens the specific port to allow only the specified service and blocks any other communication (i.e., it is a default-deny-all firewall).

Specifically, Figure 4 illustrates the operation flow of the SDP architecture during connection setup [76, 77]. In its operation, after the deployment of the SC to the SDP architecture, it can be linked to other organization services such as authentication and authorization services, device attestation, geo-location PKI and IAM services. The SSAH is also added to the SDP architecture; thus, it is authenticated first and then connected to the SC. The SSAH does not acknowledge any communications or respond to any unprovisioned request. The SC authenticates the SCIH connection; then, it authorizes the SCIH via initial policy verification to validate the authorized resources for the SCIH. After authentication of the SCIH, the SG resolves the list of authenticated SASH that the SCIH is authorized for communication. Then, the SG instructs the SASH to accept the request for communication from SCIH and initiate any particular policy for secure (encrypted) communications. This way, the SG sets the control plane for handling data plane activities. The SCIH initiated SPA to each authorized authenticated SASH; the SCIH creates a two-way encrypted connection via mTLS (i.e., both participated

stops the lateral movement of malicious actors targeting critical resource access (i.e., sensitive systems and data). The following subsection presents our proposed framework leveraging implementation strategies highlighted in this subsection as an approach to zero-trust for improving Security in SDN/NFV-enabled campus networks.

3.3.3. The Mapping of the SDN/NFV-based Infrastructure Building Blocks with SDP

Applying the application of SDP as a mechanism/approach to implementing a zero-trust strategy relies on understanding the resources/entities participating in the infrastructure. Typically, it is done via inventory, followed by mapping the transactions/interactions between these entities. Later, it allows one to decide which zero-trust enabling technology suits the application domain, accompanied by tailor-made zero-trust policies reflecting the domain context. The last step is continuous monitoring and control of the operations within the infrastructure under SDP protection. In SDN/NFV-based infrastructure, the following are essential building blocks for consideration:

1. Network Devices: Network devices in an SDN/NFV-based system are divided into the forwarding and control planes—traditional switches and routers forward traffic based on specified routing protocols and policies in the forwarding plane. A centralized controller manages the control plane, orchestrating traffic flow and regulations.
2. Virtualization: Network Function Virtualization (NFV) virtualizes traditional network functions like firewalls and load balancers and runs them as software instances on commodity hardware. This strategy allows for more efficient resource utilization and faster deployment of new services.
3. Orchestration is the management of the deployment and configuration of virtual network functions and services. It automates service delivery and optimizes resource allocation in real-time.

The following are the steps that SDP can take to protect the normal operations of campus networks' SDN/NFV-enabled infrastructure by exploiting the capabilities of its building blocks:

- The access policies and criteria for the SDN/NFV-enabled infrastructure are specified by the software-defined perimeter (SDP) controller. Policies can be set based on user role, device type, location, and other considerations.

- The SDP controller creates an invisible to unauthorized entities protective barrier around the infrastructure. The SDP gateway enforces the perimeter and is the only access point for authorized users and devices.
- The SDP gateway authenticates users and devices and determines whether they have permission to access the SDN/NFV-enabled infrastructure. Furthermore, adaptive authentication systems like multi-factor authentication improve security.
- To protect communication between authorized users or devices and the infrastructure, the SDP gateway employs encryption technologies such as TLS/SSL. This method assures that unauthorized actors cannot intercept or manipulate network data.
- The SDP continuously analyzes infrastructure access and usage to detect irregularities or potential security concerns. If a threat is recognized, the SDP controller can quarantine the user or device or take other corrective actions.

For example, in an SDN/NFV-enabled architecture, a user may require access to a particular virtual Network Function (VNF). The user authenticates themselves using the SDP gateway and enters the required credentials. The SDP gateway validates the credentials and examines the SDP controller's access policies. If the user is authorized to access that VNF, the SDP controller creates a dynamic perimeter around it that can only be accessed via the SDP gateway. TLS/SSL is used by the SDP gateway to encrypt communication between the user and the VNF. The SDP also continuously analyzes VNF access and usage for irregularities or potential security concerns. If a threat is recognized, the SDP controller can quarantine the user or device or take other corrective actions. In addition, the SDP ensures that only authorized devices or users can access the infrastructure, and all communication is encrypted, making it harder for attackers to breach the network.

3.4. Zero Trust Architecture for a Secure SDN/NFV-Enabled Campus Network

The proposed ZTA-SDP approach aims to improve the security of SDN/NFV-enabled campus network infrastructure. Incorporating ZTA mechanisms that holistically reduce attack surfaces is paramount to this aim. The ZTA-SDP mechanisms allow the framework to restrict/limit network connections and access between objects in the SDN/NFV-driven network infrastructures. Furthermore, it creates logical boundaries around the network services, limiting connections and allowing

only trusted clients to access SDP-enabled VNFs.

We design our ZTA-SDP security architecture using an adaptive policy-based system with strong authentication and authorization control. For securing communication, we use encryption and guarantee thorough integrity using TPM/vTPM. Furthermore, we use the SDP approach to cloak the SDN/NFV infrastructure, devices, and services. Figure 5 illustrates the proposed ZTA-SDP framework for a secure SDN/NFV-enabled campus network. The adopted ZTA-SDP deployment model is Enclave-Based Deployment, a modification of the device agent/gateway (a client/ server) model per NIST's special publication for ZTA[44]. In our proposed model, the gateway parts/elements may not abide on assets or in front of individual resources but reside at a resource enclave boundary instead.

Furthermore, we modified the adopted deployment model to a hybrid model to allow flexibility, as the enterprise assets may/may not have a device agent to connect to enclave gateways. The adopted ZTA-SDP approach incorporates the SDN controller and MANO, which sets up and reconfigures the network based on the decisions made by the ZTSE. For instance, after passing the SDP process for SDN's case, the clients request access via PEPs under the SDN controller component management. The clients begin sessions as untrusted. They must accumulate trust by various mechanisms, ultimately earning enough to access the service. Strong authentication affirming that a device is managed (company-owned), for instance, might gain a fair bit of trust but not sufficient to permit access to the billing system. Presenting the correct RSA token might provide a fair bit more trust, sufficient to access the billing system if coupled with the trust inferred from successful device authentication. Hereafter, we describe the proposed three-tiered system architecture components and their related functions for further clarification. The proposed system architecture comprises the user, security, and enforcement planes. Each tier comprises several components as follows:

- User Plane - The user plane facilitates system administrator duties and responsibilities, including human expertise to verify and evaluate operational and policy compliance. The tasks are not limited to overall control and system management.
- Security Plane - This plane oversees the system architecture's security aspects by deriving the context information, encompassing the monitoring information, including the environmental data fetched from the enforcement plane and the real-time instantiated system model. To this end, the trust

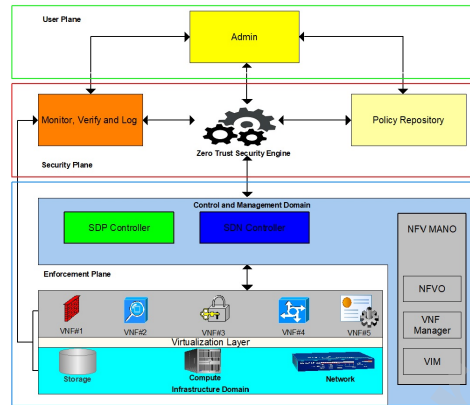


Figure 5: A Proposed ZTA-SDP for an SDN/NFV-Enabled Campus Network

parameters set derives the information with security context to form the input policy and the authorization decision. The security plane consists of three components, namely, Monitor, Validate, and Log component (MVL), Zero-Trust Security Engine (ZTSE), and Policy Repository (PR).

- MVL – This module collects data about the network asset’s current state and employs updates to software and configuration components. An MVL system gives the ZTSE data about the entity requesting access to resources/services. Such data infer information about the requestee, whether running the proper patched applications and Operating System (OS) was audited last time or whether the asset possesses some known vulnerabilities or a specific hardware security feature.
- PR - This module is responsible for policy storage. The PR stores all the policy rules referenced by the ZTSE in a version control system. This storing approach provides several benefits, including tracking policy changes over time and tracking the rationale for policy changes in the version control system. Also, it allows validation of the expected policy’s current state versus the actual/exact enforcement mechanisms.
- ZTSE – The ZTSE is a crucial system component in our architecture. It calculates the authorization decision based on available and accessible data by the ZTSE and the policy definitions crafted by the user plane (e.g., system administrators). It leverages an innovative concept in securing systems by considering the ultimate authorization decisions based on the computed

trust scores. In this regard, it is responsible for computing a trust score of the system's components using static and advanced inferred algorithms deduced from past observable behavior/ facts. The numerical trust score determines the trustworthiness of the system component/element. It enables the policy writers to concentrate on the trust level needed to access resources instead of the explicit specifications regarding what actions might degrade that trust. To this end, our ZTSE adopts a simple approach to get started by defining ad-hoc rules that score an entity's trust. For instance, a device that misses the latest software patches scores less. Similarly, an entity continuously failing to authenticate reduces its trust score. However, the ad-hoc trust scoring, depending on a statically defined set of rules, is inadequate to satisfy the desired goal of protecting against unforeseen attacks.

Consequently, in addition to applying static rules, we are working to incorporate machine learning techniques in future work to determine a scoring function. Furthermore, deciding which system components of a ZTA network to score is exciting and challenging. For example, should scores be computed separately for each entity (i.e., user, device, and workload), for the network agent as a whole, or both? In this regard, the appropriate solution is to calculate the network agent and the primary entities (i.e., user, device, and workload) that form the network agent. Then, the ZTSE leverages the computed scores to choose the correct/appropriate system component(s) and authorize based on the crafted policy. Incorporating a trust score into device and application metadata enables tractable policies to convey complete specifications, and the computed trust score captures the unknown unknowns. To this end, we have implemented algorithms A, B, and C to quantify the trust score for authorization decision-making and address the issue as mentioned above by computing both scores, namely Agent Trust Score (ATS), User Trust Score (UTS), Device Trust Score (DTS), and Workload Trust Score (WTS). Algorithm one (1) (main algorithm) starts by receiving an access request for the resource, triggering an ephemeral agent's formation. Authorization decision-making on the access request depends on the agent's trust score, computed using the entity's available information from the data stores. Algorithm two (2) computes each entity's ATS and individual scores. Depending on the resource access request, the ATS incorporates a separate entity score or altogether (i.e., user, device, and workload score). Algorithm three (3) fetches policy based on the accrued ATS. The computed score is authorized based on the crafted policy and granting access to the resource as per attributes of the accumulated ATS, otherwise ending the session. The ATS is a dynamic/adaptive numeric value reflecting

contextual resource access requirements at run-time. Hereafter are the algorithms describing our approach to quantify the trust score.

Algorithm 1 Main Algorithm

```

1: Start
2: Receive Request
3: Compute Agent Trust Score (ATS)
4: if ATS valid == false then
5:   Deny access
6: else
7:   Get policy based on ATS
8:   Grant access to Resource-based on policy
9:   while session over == false do
10:    Go to step 7
11:   end while
12: end if
13: Assign ATS = 0
14: Close connection
15: End

```

Algorithm 2 Agent Trust Score (ATS) Computation Algorithm

```

1: Procedure Compute ATS()
2: Initialization
3: UTS = DTS = WTS = 0
4: Fetch user info from DB
5: if login normal == true then
6:   Assign UTS = 1
7:   Fetch Device info from DB
8:   if device trusted == true then
9:     Set DTS = 1
10:   Fetch workload info from DB
11:   if workload valid == true then
12:     Set WTS = 1
13:     ATS = UTS + DTS + WTS
14:     Return ATS
15:   End Procedure =0

```

Algorithm 3 Algorithm to fetch policy based on ATS value

```
1: Procedure fetch relevant policy(ATS value = Computed ATS())
2: Switch (ATS value)
3: Case 1:
4: Fetch policy of ATS = 1 from DB
5: Case 2:
6: Fetch policy of ATS = 2 from DB
7: Case 3:
8: Fetch policy of ATS = 3 from DB
9: Default:
10: Deny access to the resource
11: Return policy
12: End Procedure =0
```

- The Enforcement Plane

The enforcement plane consists of an infrastructure and virtualization domain, a VNF domain, and a domain responsible for control and management.

- The Infrastructure and Virtualization Domain

This domain includes all the physical entities/machines competent in accommodating storage and computing; networking functionalities and virtualization technologies provide a layer for facilitating infrastructure as a Service (IaaS). Also, this domain comprises the network elements liable for forwarding traffic, per the controller's rules of the SDN and SDP, and security probes distributed across the infrastructure for collecting data necessary to support the monitoring services.

- VNF Domain

The VNF domain is responsible for enforcing security within network services leveraging the VNFs implemented in the virtualization infrastructure. The framework addresses specific attention to accommodating high-level security VNFs (IDS/IPS, virtual firewalls, TPM, SDP gateways, et cetera), offering the defense and threat mitigation mechanisms required by network security policies. The deployed specific mechanisms verify the trustworthiness of VNFs and monitor their key parameters continuously.

- The Control and Management Domain

This domain manages the resource usage and run-time services of security control enablers deployed atop SDN/NFV-enabled networks. A distributed set of controllers (SDN and SDP) communicates with their respective network elements to manage and control the connectivity's physical and virtual infrastructure. In addition, the NFV ETSI MANO amenable modules bolster the secure virtual security functions management in virtualized infrastructure.

- Operation of the Proposed Model

The enforcement plane consists of a control and management domain that controls and manages the SDP controller, SDN controller, and NFV MANO's networking functionality.

- SDP

The SDP follows a 'need-to-know' approach which first verifies the device's/application's identity and authenticates before granting access to the network infrastructure. It offers vital security capabilities, including software-defined abstraction, centralized management, automation, and programmability. Furthermore, it facilitates micro-segmentation of the network access by dynamically creating one-to-one connections between users and the resources. In this regard, it cloaks/blackens the application and network infrastructure; therefore, unauthorized users cannot detect/see the infrastructure [46, 47, 49, 74, 76]. Consequently, this can dramatically alleviate several network-based attacks, such as DoS/DDoS, server scanning, and MiTM, among many others [46, 47, 49, 74, 76].

- SDN

The SDN Controller deals with fundamental security techniques for SDN tasks, including traffic flow mirroring, forwarding, bandwidth reduction, dropping, et cetera. Hence, it simplifies the management of networks and enables innovation within communication networks through software-defined abstraction, centralized management, programmability, and automation capabilities.

- NFV MANO

As virtual security functionalities, our proposed framework considers the implementation of different VNFs via the NFV MANO. For instance, Virtual Switch/Router, Virtual Bandwidth control, Virtual IDS/IPS, Virtual Honeynet /Honeypot, Virtual

Firewall, Virtual Security Web Proxy, and Virtual VPN. It manages tasks, coordinates, and automates the NFV architecture. Also, this block includes VNF Manager (VNFM), Virtual Manager (VIM), and VNF Resource Orchestrator (NFVO): NFVO manages the Network Functions Virtualization Infrastructure (NFVI) resources used by VIM. VNFM manages and processes domain configurations and virtual network functions lifecycle. VIM utilizes and controls NFVI resources.

Generally, the proposed system improves the security of the SDN/NFV-enabled infrastructure by automatically deriving the context of all the access requests in a zero-trust perspective (all requests are untrusted; thus, regularly verifying all resource requests is a must). It accomplishes this through the following steps:

By applying stringent adaptive access control mechanisms, the least privilege principle, detecting threats, and cloaking the devices and infrastructure with the ZTA-SDP-based security framework, this approach can effectively secure network elements and resources by making them accessible and discoverable to only trusted and appropriate entities. Furthermore, we incorporate the computed variable trust score and situational context derived from the feedback mechanisms' analytics to evaluate resource access requests for decision-making.

The proposed framework allows fast provisioning and de-provisioning of lightweight security functionalities in a zero-trust compliant manner (as portrayed in section V). It explores and leverages managed data from available inventory and historical stores for devices discovering and assets inventory, including Security Information and Event Management (SIEM), Configuration Management Database (CMDB), and related inventory systems. Furthermore, it encrypts, logs, and audits all traffic. It uses feedback mechanisms from the inferred insights and the inventory and historical store to compute a variable trust score of all communication (resource access requests) for efficient access decision-making. It incorporates the computed variable trust scores for adaptive policy-based security management to establish the least privilege access control.

Furthermore, it extends this to adaptive policy-based segmentation (micro-segmentation) to the individual workload. While SDN/NFV capabilities ease connectivity and network management, SDP capabilities secure the connection and network infrastructure applications using the 'need-to-know' basis model. Specifically, adopting the ZTA-SDP approach produces resilient systems for most modern-day attack vectors (the following section highlights security threats within the SDN/NFV-enabled campus network and analyzes them quantitatively against the proposed model).

4. SDN/NFV-Enabled Campus Network Adversarial Model, Experimental Setup for Security Analysis and Performance Evaluation of the Proposed Model

This section presents a threat model for the SDN/NFV-enabled network, establishes an experimental setup, performs a proposed model's security analysis, and evaluates the performance overhead incurred by the ZTA-SDP-based model.

4.1. SDN/NFV-Enabled Campus Network Adversarial Model

The SDN/NFV-enabled campus network infrastructure comprises features, functionalities, or services that malicious actors can exploit as they give a broad attacking surface.

4.1.1. Attack surface 1

- Port Scan/ Network reconnaissance. Within the SDN/NFV-enabled campus infrastructure network via a multipurpose utility tool and a port scanner such as Nmap [78], a malicious actor can discover services and hosts running in a campus network, including host and port discovery. The malicious actor can exploit the network infrastructure and impact the network security posture and operations with this information.

4.1.2. Attack surface 2

- DoS/DDoS attacks. The DoS/DDoS attacks are prevalent attacks within the network infrastructure, including SDN/NFV-enabled campus networks. In this case, the malicious actor can launch the DoS/DDoS attacks internally or externally on the SDN/NFV-enabled campus network. However, Internet-facing services are more frequently subject to DoS/DDoS than internally-hosted services; in this regard, our consideration is limited to the latter and not the former. Furthermore, the impact of these attacks on network services varies from medium to high, such as unexpected service outages, traffic flow redirection to bypass security technologies, or steering traffic toward compromised systems, for the interception or modification of traffic elsewhere in the network [41]. Specifically, several well-known DDoS attacks target network services and applications (including servers and customer/employee portals) within the campus networks, including application, transport, and network layers attacks (HTTP Flood, TCP SYN, and UDP Reflection, respectively).

- **HTTP Flood Attack:** In a typical HTTP flood attack, a malicious actor sends HTTP requests that seem to originate from an actual service user (e.g., a web application). This approach can inflate the challenge using existing mitigation techniques (e.g., request rate-limiting). For instance, some HTTP floods target a particular resource, while more complicated HTTP floods tend to mimic the interaction between humans and the applications.
- **TCP SYN Attack:** In the TCP SYN flood attack, usually, a malicious actor (client) sends many SYN packets to the TCP service(e.g., a web server) but never finally sends the ACK packets to finish the three-way TCP handshakes. In this case, the server is kept waiting for a response message to the half-opened TCP connections. Ultimately, the server is depleted of its capability to admit new TCP connections. This attack prevents connections to the server for legitimate/authorized users. The attack's primary goal is not about the volume generated by the SYN traffic but instead occupying available server connections ending up with no resources for legitimate/authorized connections. SYN floods can span up to the size of hundreds of gigapits per second (Gbps).
- **UDP Reflection Attack:** UDP reflection attacks misuse UDP being a stateless protocol. Malicious actors can create a valid request packet of UDP with the source IP address listed as the attack target's IP. The malicious actor has now forged (aka, spoofed) the UDP request packet's source IP. Then, the malicious actor sends the forged source IP's UDP packet to an intermediary server. The intermediary server is used because it creates more enormous responses than the request packet, dramatically magnifying the malicious traffic transferred to the target IP address. In addition, the server is misguided into transferring its UDP response packets to the IP of the targeted victim instead of transferring them back to the attacker's IP address.

4.1.3. *Attack surface 3*

- **Malicious Use of Authorized Resources(Insider Threats).** Within the SDN/NFV-enabled campus network, an insider threat can utilize the broad network access level to the available and authorized resources to conduct malicious activities. For instance, authorized network entities (users/applications and devices) can perform intentional or accidentally unauthorized and suspicious activities/attempts such as port scanning, ping sweep, lateral movement, data exfiltration, and spreading malware, to mention a few.

4.1.4. Attack surface 4

- Use of stolen credentials. The credentials that fall on the malicious actor's hands can be used to perform malicious acts such as misuse of the authorized resource and many more. Stolen credentials (via brute force, keyloggers, phishing) or lost credentials by the user can impact the SDN/NFV-enabled campus network security posture.

4.2. Experimental Setup

The experimental setup consists of four VMs deployed in a cloud environment with another VM with Linux Ubuntu 20.04 (LTS) operating systems in a Windows 10 physical machine running VirtualBox 6.2 software. The experiment implementation comprises the SDN and SDP parts—the dedicated VM running the RYU SDN controller in the cloud and the installed Open vSwitch (OVS) inside the host machine, accomplishing the SDN part implementation. The SDP part includes a dedicated VM that runs the SDP controller module in the cloud and the other VM installed to run the developed fwknop module to represent SDP-IH on an authorized client inside the host machine. Furthermore, another VM deployed in the cloud implements a deny-all default gateway enclaving the protected services by using iptables for routing(leveraging the fwknop module with customized configurations to act as SDP AH protecting the available services). Finally, another VM implements the malicious VM to perform malicious activities against the proposed approach. We have implemented a simplified NFV Orchestrator and Virtualized Infrastructure Manager in our experimental setup by utilizing cloud infrastructure. This approach allows us to effectively deploy and manage VMs while coordinating their deployment and termination, managing the underlying cloud infrastructure, and handling resource allocation and abstraction for the deployed VMs. Table 1 shows the detailed test bench setup specifications and clarifications of the system setup and development.

The implemented SDP prototype comprises multiple components integrated to achieve the main objective of our system development. We have incorporated usability and other related metrics to meet our intended tasks. The SDP infrastructure implementation in our experimental setup is the customized version of the SDP open source from the Waverley Labs, Inc¹. It is also available via Aifruis Labs². The infrastructure comprises the SDP controller, which manages clients'

¹<https://github.com/WaverleyLabs>

²<https://github.com/aifruislabs/sdp-controller-vue-js>

Table 1: Experimental Setup Components Specifications/Descriptions

Machine	Software	Specification
Host	Linux Ubuntu 20.04 (LTS) 64-bit VirtualBox 5.2 Open vSwitch 2.5.6 Legitimate Client (fwknop module (IH configuration))	2.80GHz Intel® Core™ i7-7700HQ CPU @ 2.80GHz, 8MB Cache 16 GB DDR4 2GB GDDR5 NVIDIA GEFORCE GTX 1.0 Gbps Full Duplex Ethernet card
VM1 (SDP Controller)	Linux Ubuntu Waverly SDP controller module	1 virtual Processor 1 GB RAM 2 NICs
VM2 (Gateway)	Linux Ubuntu fwknop module (AH configuration) Services hosted behind the gateway	1 virtual Processor 2 GB RAM 3 NICs
VM3 (Malicious Actor)	Linux Ubuntu Scripts for malicious activities	1 virtual Processor 1 GB RAM 1 NIC
VM4 (SDN Controller)	CentOS 7 64-bit RYU	1 virtual Processor 4 GB RAM 2 NICs

identities and trust score computation for client authentication and verification. The developed SDP controller technologies are Vue JS for the front end, Laravel³ for the backend (i.e., API), swagger for API documentation, and MySQL version 8.0.27 for the database. Also, The SDP infrastructure possesses an SDP gateway with allocated services operating with the default deny-all policy, which accepts resources access to only authenticated and verified clients by the SDP controller. The desktop client is another component of the implemented SDP infrastructure, the lightweight software application facilitating authentication, verification, and maintaining access for the client to the protected gateway services. Generally, as shown in figures 6 (a-n) in the appendix, the tailor-made desktop SDP client fetches contextual information of the user, device, and application, facilitating the trust score computation, authentication, and verification processes. Furthermore, a scoring module is responsible for trust score computation based on the predefined metrics within the SDP controller. The core metrics in the proposed system used information from the user desktop client to give scores to users' requests based on the context upon verification are:

- **User Credentials:** This metric comprises the client username/client id and password. Upon verification, a score is earned/assigned to a request and thus allows/grants access to the appropriate allowed services based on the predefined policy.
- **Face Recognition:** This metric refers to face recognition to verify a user via a desktop client as a second factor for authentication under specific predefined conditions before giving access to certain types of services.
- **MAC Addresses:** This metric leverages a device's MAC address to verify a computer installed with the desktop SDP initiative client.
- **Geo Location:** This metric allows verifying the client's geo-location based on the radius distance specified using latitude and longitude added by the ADMIN to the client in the controller dashboard for limiting accessing specified services under specified requirements/contexts.

Updating scores depends on changes in contextual information of the user, device, and application, facilitating the trust score computation, authentication, and verification processes. This approach is enabled via continuous connection monitoring.

³<https://laravel.com/>

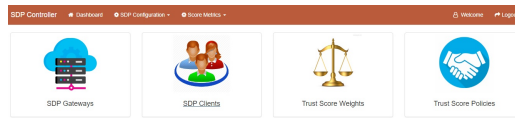


Figure 6: SDP Controller Interface Main Components

For instance, changes in network behavior (e.g., geolocation), changes in users' behavior (e.g., biometric data), and changes in the System's Reputation (e.g., Security Posture). The SDP controller interface is critical in managing the SDP gateways, clients, trust score weights, and policies. This component is essential as it simplifies the administrator management tasks through the following vital components of the system via a dashboard:

- **SDP Gateways:** This component refers to where clients will be knocking to access the services. The SDP Gateway has an iptables firewall in front of it. It blocks all requests coming to it except only those allowed by the SDP controller.
- **Clients:** This component refers to the number of people who can access certain services to specific SDP gateways such as HTTP, HTTPS, SSH, et cetera.
- **Trust Score Weights:** This component's controller facilitates assigning the weight to specific trust score metrics. For instance, a face recognition metric can have a higher trust score, such as a 50 percent score.
- **Trust Score Policies:** This component provisions contextual rules to access appropriate offered services. For instance, users with valid credentials and allowed geo-location can access HTTP, and users with validation via face recognition functionality can access SSH under specified conditions. Figure 6 shows the main components of the SDP controller interface.

With the developed SDP prototype, granting services access is after passing the authentication and validation of the contextual information via metrics previously described of the user/device/application. Then, for the key management mechanism, the system sends the request every 25 seconds to get new keys for the client. Firewall timeouts every 30 seconds after granting access to service. The module .fwknoprc is used to handle credentials for encryption and HMAC keys which then fwknop is used in port knocking to gain access to the iptables firewall. Figure

Name	Header	Payload	Protocol	Sequence	Initial	Timeout
Format:json	• data: {id: 1, name: 'client', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Key	• key: {key: 'key', value: 'value', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Format:json	• data: {id: 1, name: 'client', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Key	• key: {key: 'key', value: 'value', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Format:json	• data: {id: 1, name: 'client', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Key	• key: {key: 'key', value: 'value', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Format:json	• data: {id: 1, name: 'client', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Key	• key: {key: 'key', value: 'value', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Format:json	• data: {id: 1, name: 'client', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					
Key	• key: {key: 'key', value: 'value', ip: '192.168.1.1', port: 12345, access: '192.168.1.1/24', ...}					

Figure 7: Client Credentials Validation Process

7 shows the client credentials validation process, and algorithm four (4) describes the SDP services access and its maintenance operations.

Algorithm 4 SDP Services Access and Maintenance Operation

- 1: Client requests HMAC and ENCRYPTION KEYS to SDP Controller after every predefined time interval (25 seconds)
 - 2: **while** request received() == false **do**
 - 3: Go to step 1
 - 4: **end while**
 - 5: SDP Controller generates the HMAC and Encryption Keys
 - 6: Sends Generated KEYS to the SDP GATEWAY
 - 7: **while** file updated() == False **do**
 - 8: SDP Gateway updates FWKNOP STANZA file
 - 9: **end while**
 - 10: Sends updated stanza file with keys to controller
 - 11: SDP Controller responds with keys to the client-initiated key request
 - 12: Client updates .FWKNOPRC file with the received keys
 - 13: Client sends FWKNOP knock to the SDP Gateway server
 - 14: **if** FWKNOP knock success() == True **then**
 - 15: Grant access via IPTables and Firewall rules
 - 16: **else**
 - 17: Go to step 1
 - 18: **end if**
-

4.3. Security Analysis

This subsection provides a security analysis of the previous subsection's adversarial model against the SDN/NFV-enabled campus network's proposed security model.


```

(base) root@sdg-controller:~# nmap -sV 167.172.53.125
Starting Nmap 7.80 ( https://nmap.org ) at 2022-01-07 12:56 UTC
Nmap scan report for 167.172.53.125
Host is up (0.00076s latency).
Not shown: 994 filtered ports
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         Apache/2.4.41 ((Ubuntu))
8080/tcp   closed complex-link
9000/tcp   closed http-alt
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 11.15 seconds
(base) root@sdg-controller:~#

```

Figure 8: Port scanning with Nmap when SDP is in Disabled mode

```

(base) root@sdg-controller:~# nmap -sV 167.172.53.125
Starting Nmap 7.80 ( https://nmap.org ) at 2022-01-07 13:03 UTC
Nmap scan report for 167.172.53.125
Host is up (0.0023s latency).
All 1000 scanned ports on 167.172.53.125 are filtered
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 21.50 seconds
(base) root@sdg-controller:~#

```

Figure 9: Port scanning with Nmap when SDP is in Enabled mode

4.3.1. Attack surface 1

Port Scan/ Network reconnaissance. The proposed model's mitigation mechanism for port scanning/ network reconnaissance uses SPA and an SDP gateway to hide the network infrastructure behind the deny-all firewall and block all unauthorized network activity. Additionally, it can log and notify all connection requests for analytics. Figures 8 and 9 illustrate port scanning using Nmap⁴ in two scenarios: SDP enabled and Disabled, respectively. It is clear that with SDP disabled, services are visible and prone to port scanning and network reconnaissance attacks, while with SDP enabled, services are cloaked and not visible and only accessible to unauthorized users/devices/applications.

4.3.2. Attack surface 2

- DoS/DDoS attacks

Because our proposed ZTA-SDP model uses SPA with a deny-all firewall within the gateway, it increases the campus network's resilience to DoS/DDoS attacks. DoS/DDoS attacks are blocked with this capability and are largely ineffective in the proposed SDN/NFV-enabled campus network model. In addition, SPA drops bad packets (initiated mainly by unauthorized entities), which allows the possibility of incorporating notification and logging any network activities to the SIEM for further actions. With this approach, DDoS traffic is proactively rejected by SDP Gateways with a minimal computational load on the network infrastructure. The network services and applications (including servers and customer/employee

⁴<https://nmap.org/>

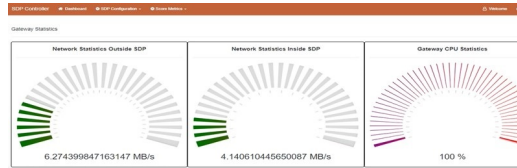


Figure 10: DDoS attacks with SDP disabled

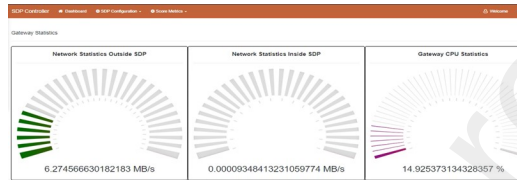


Figure 11: DDoS attacks with SDP enabled

portals) within the campus networks are well-suited to being protected from DDoS attacks by our proposed ZTA-SDP model. The target for the services mentioned above is a well-defined audience. This context allows thorough authentication of the targeted audience identities before utilizing these services. The ZTA-SDP model prevents well-known DDoS attacks, including application, transport, and network layers attacks (HTTP Flood, TCP SYN, and UDP Reflection, respectively). The ZTA-SDP leverages its three fundamental principles: making invisible services behind a deny-all SDP Gateway and authenticating users on devices before opening the firewall to establish connections; furthermore, a dynamic firewall mechanism allows ZTA-SDP to drop packets as fast as the switches serve them during a DDoS/DDoS attack. On the one hand, Figure 10 shows a network infrastructure under variable DDoS attacks with SDP disabled using the hping⁵ utility. In this case, the DoS/DDoS traffic overwhelms the network infrastructure by consuming all available resources leading to the unavailability of resources for legitimate users. On the other hand, Figure 11 shows a network infrastructure under variable DoS/DDoS attacks with SDP enabled. In this case, the DoS/DDoS traffic is proactively rejected by SDP gateways with a minimal computational load on the network infrastructure. Specifically, how the ZTA-SDP mitigates the three DDoS attacks mentioned above is described hereafter.

1. HTTP Flood Attack: The HTTP Flood attack's fundamental goal is to con-

⁵<http://hping.org/>

nect to legitimate-looking post requests using legitimate-looking devices. The most effective way to prevent this attack is to check and verify each connection thoroughly. The ZTA-SDP thwarts this attack by cloaking/hiding the target's servers from unauthorized devices. First, the attacker's botnet cannot identify the target's servers as the botnet devices in this regard are unregistered with the SDP's controller. Second, even if it could find the SDP gateway hiding the servers, the botnet cannot connect it to the SDP gateway as its devices do not have the installed SDP client, which directs any SDP controller's communication. Third, besides controlling communication, the missing SDP client contains the unique ID (per device), client certificate, and encryption keys. Fourth, the botnet can never connect because the SDP controller cannot approve/validate the single packet authorization packet (SPA) containing the authorized device's user information. Fifth, the SDP controller can never verify and match the required client information. Sixth, the SDP gateway will not grant access to the perimeter without the SDP client (with the ID, certificate, and encryption keys) installed on the botnet devices and registered with the SDP controller. Finally, the SDP gateway protecting the servers will not open up the firewall to connect to the hidden service unless authorized by the SDP controller. However, suppose the IP address becomes public, or the attacker can locate the IP address. If the attacker gets this far, the trail left by the attempted entry and the dropped packets provides evidence and data that can be analyzed to improve defenses to prosecute the attacker.

2. TCP SYN Flood Attack: The ZTA-SDP using SDP gateways provides a guard(using a firewall) that automatically drops all packets from unauthorized devices/clients and permits only packets from legitimate devices/clients into the protected target perimeter. Other defense mechanisms do not differentiate between legitimate and unauthorized devices/clients who choke good packets by providing network bandwidth and inspecting packets. This approach makes SDP-based solutions more effective for sustaining operations during the SYN Flood attacks regardless of packet rates or volumes. UDP Reflection Attack: UDP-based services cannot be secured alone since UDP is an open mechanism for delivering packets without authentication or authorization. For instance, for non-public services, services only intended to be consumed by an identifiable population of users or servers are well-suited under the ZTA-SDP protection. By putting the UDP-based services behind the SDP gateway, the campus network infrastructure is in an excellent posture to facilitate access control enforcement, allowing only au-

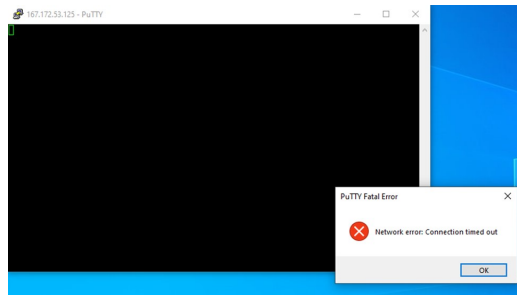


Figure 12: SDP protects resources in the malicious use of authorized resources

thorized users (or devices) to send UDP packets to the appropriate service. This approach eliminates attackers' capabilities to conduct a reflection attack utilizing the UDP-based services.

4.3.3. Attack surface 3:

- Malicious Use of Authorized Resources(Insider Threats)

The ZTA-SDP improves the SDN/NFV-enabled campus network's security posture by limiting broad access to the network infrastructure. On the one hand, authorized users access only authorized resources and nothing more per implemented policy in our proposed SDN/NFV-enabled campus network model. On the other hand, the continuous monitoring and analysis of users' activity for anomalous behavior detect and limit insider threats from accessing and using the network resource maliciously. Furthermore, through binding/mapping, a computed trust score to applications' metadata and devices allows for dynamic and adaptive policies that explicitly state requirements. Nevertheless, still, the computed trust score captures the unknown unknowns. The ZTA-SDP enforces the least privilege by limiting access to only allowed (available and visible) resources per policy and accumulated trust score. Figure 12 illustrates this scenario where the SDP restricts SSH as a service to an authorized user who failed to meet an appropriate score per a predefined security policy.

4.3.4. Attack surface 4

- Use of Stolen Credentials

Since a ZTA approach authenticates both the user/application and the device, a malicious actor must steal at least two identities to access its resources. This approach improves security posture compared to conventional approaches and



Figure 13: SDP restricts the use of stolen credentials by non-legitimate users(unauthorized VPN) via multiple factors verification of user/device/application

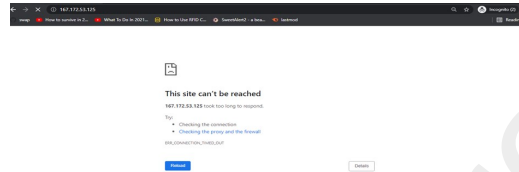


Figure 14: SDP mitigates the use of stolen credentials by non-legitimate users(incognito mode), leveraging contextual information to verify user/device/application

strategies in today's networks. The proposed ZTA-SDP-based model incorporates MFA, mTLS, and device fingerprinting to mitigate attacks associated with stolen credentials. In addition, it blocks and notifies the use of stolen credentials as it requires multiple factors before connection. In this regard, stolen passwords are insufficient for malicious actors to access network resources. Figures 13 and 14 show the multiple variable requests that failed to access the secured resource using stolen credentials after missing other metrics with necessary contextual information to verify and approve the resource access requests. Generally, the proposed approach, which adopts the need-to-know model for connectivity, is resilient and robust against many other common security threats within the SDN/NFV-enabled campus network infrastructure, including connection hijacking, man-in-the-middle, certificate forgery, and DNS poisoning. In addition, its operation mitigates the security threats mentioned above via encryption pinned certificates and no DNS (no conveying of DNS information, IP addresses, or ports of the internal network infrastructure).

4.4. Performance Evaluation

This subsection describes the performance of the proposed ZTA-SDP powered with the Trust Score Algorithm. For performance evaluation, metrics are delay, throughput, and CPU usage in two testing scenarios: ZTA-SDP enabled/disabled.

4.4.1. ATS Performance Analysis.

The proposed ATS algorithm performance results show the trivial inevitable variable computation overhead, which depends on the number and type of the

scoring metrics incorporated in the trust score algorithm. However, the request verification of these metrics is inevitable and vital in securing resources' access within modern network regular operations. Table 2 shows the computation time and size for each score metric verification request incorporated in the implemented ATS algorithm.

Furthermore, in SDP-enabled environments, the SDP components experience

Table 2: Trust score metric computation time and size

Score metric	Time(in milliseconds)	Size(in bytes)
Username and password	317	436
MAC address	242	375
Geolocation	240	329
Face recognition	3490	391

trivial startup time overhead compared to the environments without the application of the SDP. The overhead is primarily due to two additional steps: a client has to send encryption and HMAC requests, and a server has to send a request to the gateway to update Stanza. Specifically, the initiating host authentication takes 1250 milliseconds for encryption and HMAC requests. Also, the accepting host authentication time incurs an extra 455 milliseconds for a server to request the gateway to update Stanza. However, the aforementioned additional steps are not of concern within the regular SDN network. Similarly, previous research works [47, 76, 79, 80] encountered such overhead, which varies in size based on the system components' computation capability and the link speed between participated entities/components.

4.4.2. ZTA-SDP Performance Analysis

- CPU usage: The performance evaluation of the CPU usage with the proposed trust score algorithm in the ZTA-SDP shows insignificant changes from the setup without running the trust algorithm. For single(minimum) and multiple (maximum) requests, the CPU usage varies insignificantly and thus proves the viability of the proposed algorithm in an actual deployment environment. On the one hand, with the single(minimum) request in the SDP disabled mode, the CPU usage reached a maximum percentage of 21.68 (Mean=14.97, Standard deviation(STD)=3.00, and Variance=9.00). On the other hand, with the multiple(maximum) requests in the SDP dis-

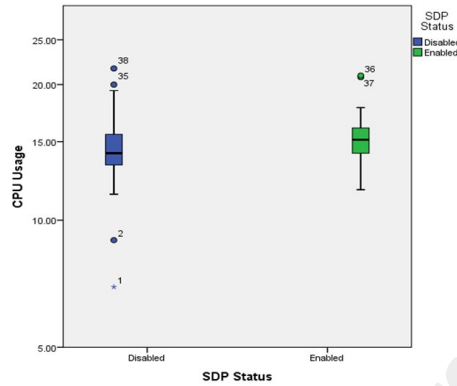


Figure 15: SDP gateway CPU usage percentage with a minimum number of requests

abled mode, the CPU usage reached a maximum percentage of 21.68 (Mean=14.86, Standard deviation(STD)=2.87, and Variance=8.23).

On the other hand, with the single(minimum) request in the SDP-enabled mode, the CPU usage reached a maximum percentage of 20.92 (Mean=15.39, Standard deviation(STD)= 2.37, and Variance=5.63). With the multiple(maximum) requests in the SDP-enabled mode, the CPU usage reached a maximum percentage of 20.92 (Mean=14.48, Standard deviation(STD)= 2.44, and Variance=5.93). Figures 15 and 16 show the CPU usage percentage when the SDP is enabled and disabled for single(minimum) and multiple (maximum) requests, respectively. The CPU usage percentage shows that with SDP setup enabled varies insignificantly when disabled.

- Delay: Figures 17 and 18 depict delays between core peers(a client, SDP controller, and SDP gateway) in ZTA-SDP for disabled and enabled modes, respectively. The delay between peers in the implemented ZTA-SDP was insignificant as we measured it with the ZTA-SDP setup in the disabled and enabled mode. The delay is trivial as the route creation process between the client, gateway, and services happens only once after establishing the route information in the SDN switch flow tables; the already established route can serve similar requests. There is no need to repeat the route creation process for other similar requests. Instead, the delay bottlenecks are the processing capabilities and the connection links between connected peers.

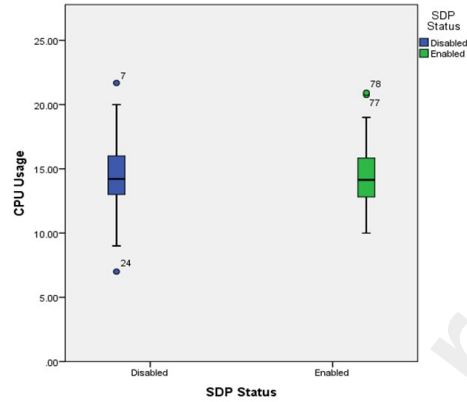


Figure 16: SDP gateway CPU usage percentage with a maximum number of requests

```

root@alfruits-VirtualBox:~# ping 178.62.76.239 -c 4
PING 178.62.76.239 (178.62.76.239) 56(84) bytes of data.
64 bytes from 178.62.76.239: icmp_seq=1 ttl=50 time=187 ms
64 bytes from 178.62.76.239: icmp_seq=2 ttl=50 time=197 ms
64 bytes from 178.62.76.239: icmp_seq=3 ttl=50 time=177 ms
64 bytes from 178.62.76.239: icmp_seq=4 ttl=50 time=195 ms
--- 178.62.76.239 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 177.104/189.074/197.180/7.935 ms
root@alfruits-VirtualBox:~# ping 167.172.53.125 -c 4
PING 167.172.53.125 (167.172.53.125) 56(84) bytes of data.
64 bytes from 167.172.53.125: icmp_seq=1 ttl=50 time=186 ms
64 bytes from 167.172.53.125: icmp_seq=2 ttl=50 time=185 ms
64 bytes from 167.172.53.125: icmp_seq=3 ttl=50 time=184 ms
64 bytes from 167.172.53.125: icmp_seq=4 ttl=50 time=236 ms
--- 167.172.53.125 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 184.268/197.849/236.294/22.204 ms

```

Figure 17: Delays between core peers in ZTA-SDP disabled mode

```

root@alfruits-VirtualBox:~# ping 167.172.53.125 -c 4
PING 167.172.53.125 (167.172.53.125) 56(84) bytes of data.
64 bytes from 167.172.53.125: icmp_seq=1 ttl=50 time=183 ms
64 bytes from 167.172.53.125: icmp_seq=2 ttl=50 time=182 ms
64 bytes from 167.172.53.125: icmp_seq=3 ttl=50 time=197 ms
64 bytes from 167.172.53.125: icmp_seq=4 ttl=50 time=190 ms
--- 167.172.53.125 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 182.090/188.112/197.339/6.191 ms
root@alfruits-VirtualBox:~# ping 178.62.76.239 -c 4
PING 178.62.76.239 (178.62.76.239) 56(84) bytes of data.
64 bytes from 178.62.76.239: icmp_seq=1 ttl=50 time=223 ms
64 bytes from 178.62.76.239: icmp_seq=2 ttl=50 time=180 ms
64 bytes from 178.62.76.239: icmp_seq=3 ttl=50 time=204 ms
64 bytes from 178.62.76.239: icmp_seq=4 ttl=50 time=181 ms
--- 178.62.76.239 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 180.156/196.842/222.813/17.730 ms

```

Figure 18: Delays between core peers in ZTA-SDP enabled mode


```

root@alfruits-VirtualBox:~# iperf -c 167.172.53.125
Client connecting to 167.172.53.125, TCP port 5001
TCP window size: 45.0 KByte (default)
[ 3] local 192.168.43.141 port 53258 connected with 167.172.53.125 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0-10.0 sec  8.12 Mbytes  6.53 Mbits/sec
root@alfruits-VirtualBox:~# iperf -c 178.62.76.239
Client connecting to 178.62.76.239, TCP port 5001
TCP window size: 45.0 KByte (default)
[ 3] local 192.168.43.141 port 37032 connected with 178.62.76.239 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0-10.3 sec  7.62 Mbytes  6.22 Mbits/sec

```

Figure 19: Bandwidth between core peers in ZTA-SDP Disabled mode

```

root@alfruits-VirtualBox:~# iperf -c 167.172.53.125
Client connecting to 167.172.53.125, TCP port 5001
TCP window size: 45.0 KByte (default)
[ 3] local 192.168.43.141 port 53246 connected with 167.172.53.125 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0-10.0 sec  8.38 Mbytes  6.51 Mbits/sec
root@alfruits-VirtualBox:~# iperf -c 178.62.76.239
Client connecting to 178.62.76.239, TCP port 5001
TCP window size: 45.0 KByte (default)
[ 3] local 192.168.43.141 port 37014 connected with 178.62.76.239 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3] 0.0-10.2 sec  7.88 Mbytes  6.29 Mbits/sec

```

Figure 20: Bandwidth between core peers in ZTA-SDP enabled mode

- **Bandwidth:** The ZTA-SDP deployment does not significantly affect the infrastructure bandwidth. The bandwidth was recorded between the client and the server at the SDP gateway using the `iperf6` utility installed in the authorized client and the service server. As depicted in figures 18 and 19, with SDP disabled and enabled, respectively, it does not affect either peer's connection within the infrastructure. Instead, the bandwidth bottlenecks are the network interfaces, connection links, and processing capabilities of the connected peers.

4.4.3. A Summary Discussion on Performance Overhead.

The proposed ZTA-SDP model for SDN/NFV-enabled campus network infrastructure possesses trivial communication overhead from the connection setup time of the SPA client and the gateway before connection to the controller compared to the architecture without SDP. Specifically, adopting SDP incurs a higher connection setup time due to the required firewall rules and the SPA packet's decryption and verification process before the client connection acceptance. Contrarily, none of these processes and steps happen for the case without SDP, leading to a notably faster connection process. Also, a trivial delay is incurred from running the SDP modules in the SDN/NFV-enabled network infrastructure contrasted with the network infrastructure without SDP. A one-time flow control delay is necessary for finding the correct flow via the gateway between the SDP controller and the client. This flow control delay occurs when a device/client tries communicating with an-

⁶<https://iperf.fr/>

other device/server via an SDN switch via the gateway. The SDN switch will first look in its flow tables and search to locate the destination route; if the switch fails to discover the correct/proper flow, it will initiate the flow control sequence by dispatching to the controller a Packet-in message. The delay is trivial as the route creation process between the client, gateway, and services happens only once after establishing the route information in the SDN switch flow tables; there is no need to repeat the route creation process for other similar requests. The same already established route can serve similar requests.

5. ZERO-TRUST COMPLIANT NETWORK SECURITY FUNCTIONS IN SDN/NFV-ENABLED CAMPUS NETWORK INFRASTRUCTURE

This section briefly highlights three non-exhaustive but possible implementations of zero-trust compliant network security functions in the SDN/NFV infrastructure and gives a summary on best practices approach in accomplishing this objective. The first use case is the VM-based Trusted Platform Module (TPM) for attesting network functions and services. The other two use cases explore Unikernel Network Functions (UNFs) run in the virtual machine (aka vUNFs), which justify its feasibility with a zero-trust architecture approach. The UNFs run on virtual machines. To this end, we summarize the implementations of the zero-trust compliant unikernels for stateful firewalls and honeypots as IDS. In dynamic and heterogeneous SDN/NFV-enabled infrastructure, leveraging unikernels network security functions can be significant in achieving performance results comparable to the leading container technologies and offering the VMs' security level. Furthermore, the vUNFs fit correctly into application of trending paradigms, such as the mobile edge [81] and vCPE [82] scenarios.

5.1. vTPM for SDN/NFV-Enabled Infrastructure

Like hardware TPM chips, a virtual TPM (vTPM) holds trust credentials, identity credentials, and encryption operations (including integrity measurement). The hardware TPM refers to a cryptographic coprocessor to accommodate the secure storage of credentials, including keys. Virtualizing the TPM is essential to enable its capabilities to all VMs running on a platform. The vTPM provides a virtual Root-of-Trust (RoT) for VMs on business cloud platforms [83]. Rather than employing a physical NVRAM, alternatively, vTPM utilizes a file, represented as an NVRAM file (encrypted using virtual machine encryption), to hold secret keys (as its secure storage). VMware, Microsoft, and Google have already embraced

vTPMs in their deployed cloud platforms. The vTPM accomplishes similar functions as a TPM but does cryptographic coprocessor capabilities within the software. For instance, to protect the critical security modules and vTPM, leveraging the SGX in the cloud, the authors' research work in [84] proposed an efficient and secure software-based vTPM (SvTPM). It addresses many issues, including trust establishment between the SGX platform and vTPMs, NVRAM replacement attacks, rollback attacks, and a fine-grained trusted clock. The proposed SvTPM performs better and implements substantial run-time isolation protection for vTPMs within the cloud. The performance evaluation results prove that the SvTPM performs better than the vTPMs under physical TPM protection. In addition, the SvTPM's launch time is 2600 times faster than vTPMs based on TPM hardware. In this regard, it can be a great advantage to leverage the SDN/NFV-enabled infrastructure by orchestrating lightweight and secure vTPMs to comply with the ZTA strategy.

5.2. *Zero-trust IDS Compliant*

Unikernels facilitate composable and on-demand security functions efficiently. Its modularity allows for more free code updates. To this end, unikernels allow implementing the IDS on a VNF on-demand basis either reactively or proactively as per the effective monitoring network security policies. The Unikernel IDS (UIDS) shows excellent potential by performing better isolation, resource efficiency, and a small memory footprint without losing the security attributes[85]. For instance, a signature-based, minimalistic UIDS delivers similar detection capabilities of commonly accepted IDSs (e.g., Snort) while utilizing 8x less RAM and 2-3 times less CPU [86]. This approach is extensible to the hybrid detection methods, which support signature-based intrusion detection leveraging rule-based methods and machine learning-powered anomaly detection to detect unknown attacks (including Zero(0)-day attacks). Therefore, intrusion detection and monitoring strategies and policies should include detection filters, signatures, and related actions (for example, rise stats, logs, and alerts).

5.3. *Zero-trust Firewall Compliant*

For firewalling, implementing a vUNF for a stateful firewall achieves performance results comparable to the container-based firewalls while providing a significantly higher security level[85]. The firewall rules are created or destroyed as Unikernels. Doing layer two (L2) filtering with high computation capability is possible, thus complying with the zero-trust philosophy and strategy. Using

vUNFs offers the benefits mentioned above—however, it is worth trading off between vUNF implementation with more rules (e.g., a firewall with about 1k rules) against memory utilization and time consumption[85]. Fortunately, it is rare to find firewalls with more than 1k rules. Therefore, filtering any traffic destined for or leaving a VM is feasible, including traffic between a VM and a physical host, traffic between two physical hosts, and traffic between two VMs.

5.4. Summary

In today's world, implementing a zero-trust compliant, lightweight network security system in SDN/NFV-enabled campus network infrastructure is challenging. It involves integrating security functions, managing dynamic environments, optimizing resource utilization, and achieving comprehensive visibility and control over network traffic. However, innovative technologies and practices can be used to address these challenges, such as standardized protocols, dynamic policies, adaptive security measures, lightweight security functions, virtualization techniques, and advanced traffic monitoring and analysis tools. By addressing these challenges, campus networks can strengthen their security posture while maintaining scalability and performance.

An intelligent and adaptive security orchestration system can be implemented to enhance campus network security further. This system uses machine learning and artificial intelligence to analyze historical and real-time network data, identify anomalies, and assess device behaviors. By leveraging SDN/NFV technologies, the system dynamically configures and deploys lightweight security functions, such as intrusion detection systems, traffic filtering, encryption, and access control mechanisms. This approach enhances security posture and contributes to resource efficiency by dynamically allocating security functions based on network demand. Real-time visibility and reporting enable quick mitigation of security threats and compliance violations. Implementing these measures can significantly improve campus network security and protect against potential threats.

6. Conclusion

In this paper, the zero-trust architecture proposed to improve the SDN/NFV-enabled campus network's security holistically. We use the infrastructure cloaking/blackening capability of the ZTA-SDP approach to address inherent security flaws within the SDN/NFV-enabled campus network infrastructure. We also leverage information from existing inventory mechanisms to compute the variable trust score for policy decision-making on the requests to access any securable object

type. In this way, we improve our approach's effectiveness by minimizing the number of policies generated and the accuracy as we fetch only relevant information for permission based on the context and situational of the resource/service access requests. Additionally, we have briefly discussed the threat model and analyzed the proposed model's security and performance. Lastly, we accentuate the ZTA and SDN/NFV synergy tools and implementations to facilitate threats detection and cloaking devices and infrastructure. Our proposed model also is extensible to incorporate evaluation and feedback mechanisms for the decision made already. To this end, we believe that our proposed idea can improve security in SDN/NFV-enabled campus networks. We aim to incorporate efficient machine learning algorithms for threat intelligence and computation of the trust score for future work.

References

- [1] S. Katukam, M. Cohn, P. Kantak, S. Nathan, K. Poodari, T. Tsou, Sdn in the campus environment, ONF Solution Brief, September (2013).
- [2] C.-H. Liu, Y.-T. Yeh, The study of sdn for campus security, in: 2015 IEEE International Conference on Consumer Electronics-Taiwan, IEEE, 2015, pp. 428–429.
- [3] D. Dhakal, B. Pradhan, S. Dhimal, Campus network using software defined networking, International Journal of Computer Applications 975 (2016) 8887.
- [4] F. Liu, G. Kibalya, S. Santhosh Kumar, P. Zhang, Challenges of traditional networks and development of programmable networks, in: Software defined internet of everything, Springer, 2021, pp. 37–61.
- [5] D. Nadig, B. Ramamurthy, Securing large-scale data transfers in campus networks: Experiences, issues, and challenges, in: Proceedings of the ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization, 2019, pp. 29–32.
- [6] W. Brockelsby, R. Dutta, Augmenting campus wireless architectures with sdn, in: 2023 International Conference on Computing, Networking and Communications (ICNC), IEEE, 2023, pp. 137–142.

- [7] M. Emmelmann, M. Corici, F. Eichhorn, M. Hauswirth, T. Magedanz, Open 5g campus networks: key drivers for 6g innovations, *e & i Elektrotechnik und Informationstechnik* 139 (7) (2022) 589–600.
- [8] M. Häberle, B. Steinert, M. Menth, Firewall-as-a-service for campus networks based on p4-sfc, *Electronic Communications of the EASST* 80 (2021).
- [9] W. Brockelsby, R. Dutta, Traffic analysis in support of hybrid sdn campus architectures for enhanced cybersecurity, in: *2021 24th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)*, IEEE, 2021, pp. 41–48.
- [10] D. Scotece, A. Noor, L. Foschini, A. Corradi, 5g-kube: Complex telco core infrastructure deployment made low-cost, *IEEE Communications Magazine* (2023).
- [11] Y. Li, M. Chen, Software-defined network function virtualization: A survey, *IEEE Access* 3 (2015) 2542–2553.
- [12] D. Kreutz, F. M. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: A comprehensive survey, *Proceedings of the IEEE* 103 (1) (2014) 14–76.
- [13] W. Stallings, *Foundations of modern networking: SDN, NFV, QoE, IoT, and Cloud*, Addison-Wesley Professional, 2015.
- [14] A. A. Barakabitze, A. Ahmad, R. Mijumbi, A. Hines, 5g network slicing using sdn and nfv: A survey of taxonomy, architectures and future challenges, *Computer Networks* 167 (2020) 106984.
- [15] C. Lorenz, D. Hock, J. Scherer, R. Durner, W. Kellerer, S. Gebert, N. Gray, T. Zinner, P. Tran-Gia, An sdn/nfv-enabled enterprise network architecture offering fine-grained security policy enforcement, *IEEE communications magazine* 55 (3) (2017) 217–223.
- [16] F. Marzouk, J. P. Barraca, A. Radwan, On energy efficient resource allocation in shared rans: Survey and qualitative analysis, *IEEE Communications Surveys & Tutorials* 22 (3) (2020) 1515–1538.
- [17] M. S. Bonfim, K. L. Dias, S. F. Fernandes, Integrated nfv/sdn architectures: A systematic literature review, *ACM Computing Surveys (CSUR)* 51 (6) (2019) 1–39.

- [18] S. H. A. Kazmi, F. Qamar, R. Hassan, K. Nisar, B. S. Chowdhry, Survey on joint paradigm of 5g and sdn emerging mobile technologies: Architecture, security, challenges and research directions, *Wireless Personal Communications* (2023) 1–48.
- [19] D. Zou, Y. Lu, B. Yuan, H. Chen, H. Jin, A fine-grained multi-tenant permission management framework for sdn and nfv, *IEEE Access* 6 (2018) 25562–25572.
- [20] A. F. Murillo, S. J. Rueda, L. V. Morales, Á. A. Cárdenas, Sdn and nfv security: challenges for integrated solutions, *Guide to Security in SDN and NFV: Challenges, Opportunities, and Applications* (2017) 75–101.
- [21] D. B. Hoang, S. Farahmandian, Security of software-defined infrastructures with sdn, nfv, and cloud computing technologies, *Guide to Security in SDN and NFV: Challenges, Opportunities, and Applications* (2017) 3–32.
- [22] Q. Li, X. Deng, Z. Liu, Y. Yang, X. Zou, Q. Wang, M. Xu, J. Wu, Dynamic network security function enforcement via joint flow and function scheduling, *IEEE Transactions on Information Forensics and Security* 17 (2022) 486–499.
- [23] X. Chen, C. Wu, X. Liu, Q. Huang, D. Zhang, H. Zhou, Q. Yang, M. K. Khan, Empowering network security with programmable switches: A comprehensive survey, *IEEE Communications Surveys & Tutorials* (2023).
- [24] R. Cziva, S. Jouet, K. J. White, D. P. Pazaros, Container-based network function virtualization for software-defined networks, in: *2015 IEEE symposium on computers and communication (ISCC)*, IEEE, 2015, pp. 415–420.
- [25] Y.-D. Lin, P.-C. Lin, C.-H. Yeh, Y.-C. Wang, Y.-C. Lai, An extended sdn architecture for network function virtualization with a case study on intrusion prevention, *IEEE Network* 29 (3) (2015) 48–53.
- [26] J. Deng, H. Hu, H. Li, Z. Pan, K.-C. Wang, G.-J. Ahn, J. Bi, Y. Park, Vanguard: An nfv/sdn combination framework for provisioning and managing virtual firewalls, in: *2015 IEEE Conference on Network Function Virtualization and Software Defined Network (NFV-SDN)*, IEEE, 2015, pp. 107–114.
- [27] C. Xu, H. Lin, Y. Wu, X. Guo, W. Lin, An sdnfv-based ddos defense technology for smart cities, *IEEE Access* 7 (2019) 137856–137874.

- [28] M. C. Dacier, H. König, R. Cwalinski, F. Kargl, S. Dietrich, Security challenges and opportunities of software-defined networking, *IEEE Security & Privacy* 15 (2) (2017) 96–100.
- [29] D. Ageyev, O. Bondarenko, W. Alfroukh, T. Radivilova, Provision security in sdn/nfv, in: 2018 14th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TC-SET), IEEE, 2018, pp. 506–509.
- [30] F. Liu, G. Kibalya, S. Santhosh Kumar, P. Zhang, Challenges of traditional networks and development of programmable networks, in: *Software defined internet of everything*, Springer, 2021, pp. 37–61.
- [31] O. Tr, Principles and practices for securing software-defined networks, Open Networking Foundation (2015).
- [32] O. N. F. (ONF), Threat analysis for the sdn architecture, Technical Report 530, v1. 0 (2016).
- [33] S. Scott-Hayward, G. O’Callaghan, S. Sezer, Sdn security: A survey, in: 2013 IEEE SDN For Future Networks and Services (SDN4FNS), IEEE, 2013, pp. 1–7.
- [34] J. C. C. Chica, J. C. Imbachi, J. F. B. Vega, Security in sdn: A comprehensive survey, *Journal of Network and Computer Applications* 159 (2020) 102595.
- [35] I. Alsmadi, D. Xu, Security of software defined networks: A survey (2015).
- [36] I. Ahmad, S. Namal, M. Ylianttila, A. Gurtov, Security in software defined networks: A survey, *IEEE Communications Surveys & Tutorials* 17 (4) (2015) 2317–2346.
- [37] S. Shin, G. Gu, Attacking software-defined networks: A first feasibility study, in: Proceedings of the second ACM SIGCOMM workshop on Hot topics in software defined networking, 2013, pp. 165–166.
- [38] D. Kreutz, F. M. Ramos, P. Verissimo, Towards secure and dependable software-defined networks, in: Proceedings of the second ACM SIGCOMM workshop on Hot topics in software defined networking, 2013, pp. 55–60.
- [39] S. Lal, T. Taleb, A. Dutta, Nfv: Security threats and best practices, *IEEE Communications Magazine* 55 (8) (2017) 211–217.

- [40] M. Bursell, A. Dutta, H. Lu, M. Oadini, K. Roemer, K. Sood, M. Wong, P. Wörndle, Network functions virtualisation (nfv), nfv security, security and trust guidance, v. 1.1. 1, Eur. Telecommun. Standards Inst., Sophia Antipolis, France, Tech. Rep. gs nfv-sec 3 (2014).
- [41] M. Pattaranantakul, R. He, Q. Song, Z. Zhang, A. Meddahi, Nfv security survey: From use case driven threat analysis to state-of-the-art countermeasures, *IEEE Communications Surveys & Tutorials* 20 (4) (2018) 3330–3368.
- [42] A. Aljuhani, T. Alharbi, Virtualized network functions security attacks and vulnerabilities, in: 2017 IEEE 7th Annual Computing and Communication Workshop and Conference (CCWC), IEEE, 2017, pp. 1–4.
- [43] Z. Zhang, A. Meddahi, Security in network functions virtualization, Elsevier, 2017.
- [44] L. Brotherston, A. Berlin, Defensive security handbook: best practices for securing infrastructure, "O'Reilly Media, Inc.", 2017.
- [45] E. Haleplidis, D. Joachimpillai, J. H. Salim, D. Lopez, J. Martin, K. Pentikousis, S. Denazis, O. Koufopavlou, Forces applicability to sdn-enhanced nfv, in: 2014 Third European workshop on software defined networks, IEEE, 2014, pp. 43–48.
- [46] A. Moubayed, A. Refaey, A. Shami, Software-defined perimeter (sdp): State of the art secure solution for modern networks, *IEEE network* 33 (5) (2019) 226–233.
- [47] A. Sallam, A. Refaey, A. Shami, On the security of sdn: A completed secure and scalable framework using the software-defined perimeter, *IEEE access* 7 (2019) 146577–146587.
- [48] M. Lefebvre, D. W. Engels, S. Nair, On sdpn: Integrating the software-defined perimeter (sdp) and the software-defined network (sdn) paradigms, in: 2022 IEEE Conference on Communications and Network Security (CNS), IEEE, 2022, pp. 353–358.
- [49] J. Singh, A. Refaey, A. Shami, Multilevel security framework for nfv based on software defined perimeter, *IEEE Network* 34 (5) (2020) 114–119.

- [50] K. DelBene, M. Medin, R. Murray, The road to zero trust (security), DIB Zero Trust White Paper 9 (2019).
- [51] S. Rose, O. Borchert, S. Mitchell, S. Connelly, Zero trust architecture, Tech. rep., National Institute of Standards and Technology (2020).
- [52] J. Garbis, J. W. Chapman, Zero Trust Security: An Enterprise Guide, Springer, 2021.
- [53] Z. Tang, Research on security architecture design of smart campus platform based on sdn and active defense technology, in: Second International Conference on Digital Society and Intelligent Systems (DSInS 2022), Vol. 12599, SPIE, 2023, pp. 450–456.
- [54] G. Guo, J. Zhang, Z. Ma, Intrusion prevention with attack traceback and software-defined control plane for campus networks, Computer Science and Information Systems 18 (3) (2021) 867–891.
- [55] P. Khani, M. Sharbaf, M. Beheshti, S. Faraji, Campus network security: Threats, analysis and strategies, in: 2018 International Conference on Computational Science and Computational Intelligence (CSCI), IEEE, 2018, pp. 781–787.
- [56] S. Zheng, Z. Li, B. Li, Campus network security defense strategy, in: 2017 International Conference on Mechanical, Electronic, Control and Automation Engineering (MECAE 2017), Atlantis Press, 2017, pp. 356–359.
- [57] L. Kumari, S. Debbarma, R. Shyam, Security problems in campus network and its solutions, International Journal of Advanced Engineering & Application 1 (1) (2011) 98–101.
- [58] N. F. S. De Sousa, D. A. L. Perez, R. V. Rosa, M. A. Santos, C. E. Rothenberg, Network service orchestration: A survey, Computer Communications 142 (2019) 69–94.
- [59] V.-G. Nguyen, Y. Kim, Sdn-based enterprise and campus networks: A case of vlan management., J. Inf. Process. Syst. 12 (3) (2016) 511–524.
- [60] K. Venkatraman, V. Parthasarathy, S. S. Kumaar, M. Jayalakshmi, Supervision of network through software defined networking, in: International Conference on Information Communication and Embedded Systems (ICES2014), IEEE, 2014, pp. 1–7.

- [61] F. Kuliesius, V. Dangovas, Sdn enhanced campus network authentication and access control system, in: 2016 Eighth International Conference on Ubiquitous and Future Networks (ICUFN), IEEE, 2016, pp. 894–899.
- [62] A. Lara, Using software-defined networking to improve campus, transport and future internet architectures, The University of Nebraska-Lincoln, 2015.
- [63] A. Lara, B. Ramamurthy, Opensec: Policy-based security using software-defined networking, IEEE transactions on network and service management 13 (1) (2016) 30–42.
- [64] P. Goransson, C. Black, Sdn in other environments (2014).
- [65] C. Buck, C. Olenberger, A. Schweizer, F. Völter, T. Eymann, Never trust, always verify: A multivocal literature review on current knowledge and research gaps of zero-trust, Computers & Security 110 (2021) 102436.
- [66] R. Ward, B. Beyer, Beyondcorp: A new approach to enterprise security (2014).
- [67] B. Osborn, J. McWilliams, B. Beyer, M. Saltonstall, Beyondcorp: Design to deployment at google (2016).
- [68] M. Shore, S. Zeadally, A. Keshariya, Zero trust: the what, how, why, and when, Computer 54 (11) (2021) 26–35.
- [69] C. Cunningham, J. Blankenship, S. Balaouras, R. Murphy, M. Cyr, The zero trust extended (ztx) ecosystem, Forrester, Cambridge, MA (2018).
- [70] W.-L. Chin, H.-A. Ko, N.-W. Chen, P.-W. Chen, T. Jiang, Securing nfv/sdn iot using vnfs over a compute-intensive hardware resource in nfvi, IEEE Network (2023).
- [71] A. Liroy, T. Su, A. L. Shaw, H. Attak, D. R. Lopez, A. Pastor, Trust in sdn/nfv environments, Guide to Security in SDN and NFV: Challenges, Opportunities, and Applications (2017) 103–124.
- [72] K. K. Karmakar, V. Varadharajan, U. Tupakula, M. Hitchens, Towards a dynamic policy enhanced integrated security architecture for sdn infrastructure, in: NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium, IEEE, 2020, pp. 1–9.

- [73] G. Evan, B. Doug, Zero trust networks: Building secure systems in untrusted networks, Posts and Telecommunications Press, Beijing (2019).
- [74] J. Koilpillai, Software defined perimeter (sdp) a primer for cios, Waverley Labs LLC 276 (2017).
- [75] E. L. R. Lucion, R. C. Nunes, Software defined perimeter: improvements in the security of single packet authorization and user authentication, in: 2018 XLIV Latin American Computer Conference (CLEI), IEEE, 2018, pp. 708–717.
- [76] P. Kumar, A. Moubayed, A. Refaey, A. Shami, J. Koilpillai, Performance analysis of sdp for secure internal enterprises, in: 2019 IEEE Wireless Communications and Networking Conference (WCNC), IEEE, 2019, pp. 1–6.
- [77] A. Sallam, A. Refaey, A. Shami, Securing smart home networks with software-defined perimeter, in: 2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC), IEEE, 2019, pp. 1989–1993.
- [78] G. F. Lyon, Nmap network scanning: The official Nmap project guide to network discovery and security scanning, Insecure, 2009.
- [79] T. Lukaseder, M. Halter, F. Kargl, Context-based access control and trust scores in zero trust campus networks, SICHERHEIT 2020 (2020).
- [80] K. K. Karmakar, V. Varadharajan, S. Nepal, U. Tupakula, Sdn-enabled secure iot architecture, IEEE Internet of Things Journal 8 (8) (2020) 6549–6564.
- [81] G. ETSI, Mobile edge computing (mec); deployment of mobile edge computing in an nfv environment, ETSI ISG (2018).
- [82] N. F. Virtualisation, Etsi industry specification group (isg), network functions virtualisation (nfv); nfv performance & portability best practises, Tech. rep., Tech. report, ETSI, 2014. 6.
- [83] D. Liu, J. Lee, J. Jang, S. Nepal, J. Zic, A cloud architecture of virtual trusted platform modules, in: 2010 IEEE/IFIP International Conference on Embedded and Ubiquitous Computing, IEEE, 2010, pp. 804–811.

- [84] J. Wang, C. Fan, J. Wang, Y. Cheng, Y. Zhang, W. Zhang, P. Liu, H. Hu, Svtpm: A secure and efficient vtpm in the cloud, arXiv preprint arXiv:1905.08493 (2019).
- [85] T. Kurek, Unikernel network functions: A journey beyond the containers, IEEE Communications Magazine 57 (12) (2019) 15–19.
- [86] V. Cozzolino, N. Schwellnus, J. Ott, A. Y. Ding, Uids: Unikernel-based intrusion detection system for the internet of things, in: DISS 2020-Workshop on Decentralized IoT Systems and Security, 2020.